

Steganography in Spatial Domain (Non-Blind)



Cluster Innovation Centre

University of Delhi

Adarsh, Kapish, Nitin, Shantanu and Swarnim

December 2023

ACKNOWLEDGEMENT

We would like to express our sincere gratitude and appreciation to **Dr. Anjani Kumar Verma** for his invaluable guidance and mentorship throughout the development of our report entitled "**Steganography in Spatial Domain (Non-Blind)**" during the academic year **2023-2024**. His expertise, encouragement, and insightful feedback have been instrumental in shaping and enhancing the quality of this research endeavor. We are truly thankful for the opportunity to learn and grow under his supervision, and we are indebted for the knowledge and skills acquired during this mentorship. Dr. Anjani's commitment to academic excellence and support has significantly contributed to the success of this project, and we are honored to have had the privilege of being mentored by such a dedicated and knowledgeable professional.

Certificate of Originality

The work embodied in this report entitled “**Steganography in Spatial Domain (Non-Blind)**” has been carried out by **Adarsh, Kapish, Nitin, Shantanu, and Swarnim** for the paper “**Steganography in Spatial Domain (Non-Blind)**”. We declare that the work and language included in this project report is free from any kind of plagiarism.

(Name of the Mentor)

(Signature of the Mentor)

Adarsh

Kapish

Nitin

Shantanu

Swarnim

1. ABSTRACT

Steganography- a technique of hiding an information (payload) in an image (cover image). It is used to hide crucial information in an image or in any kind of digital or physical object. Steganography is further divided into two large domains- the Spatial domain and the Transform domain. Our paper only revolves around the non-blind spatial domain. We have proposed a hybrid algorithm by using two pre-existing algorithms of image hiding and tried to provide a better result accuracy through it (LSB method and Spread Spectrum Steganography). This system hides and recover messages of substantial length within digital imagery while maintaining the original image size and dynamic range. We have calculated and compared the MSE and PSNR values of our hybrid algorithm with an online steganography tool that uses AES method for encryption.

KEYWORDS: Steganography, LSB, Spread Spectrum Steganography, MSE (Mean Square Error), PSNR (Peak signal to noise ratio) and AES(Advanced Encryption Standard).

2. INTRODUCTION

Steganography is the art of concealing information(payload) in a multimedia file (Image, Video, audio, etc.). It is closely related to cryptography. But their principles are entirely different. Cryptography involves converting the message into ciphertext to secure it. Whereas, Steganography involves hiding the very existence of a message by embedding the message into a multimedia file.

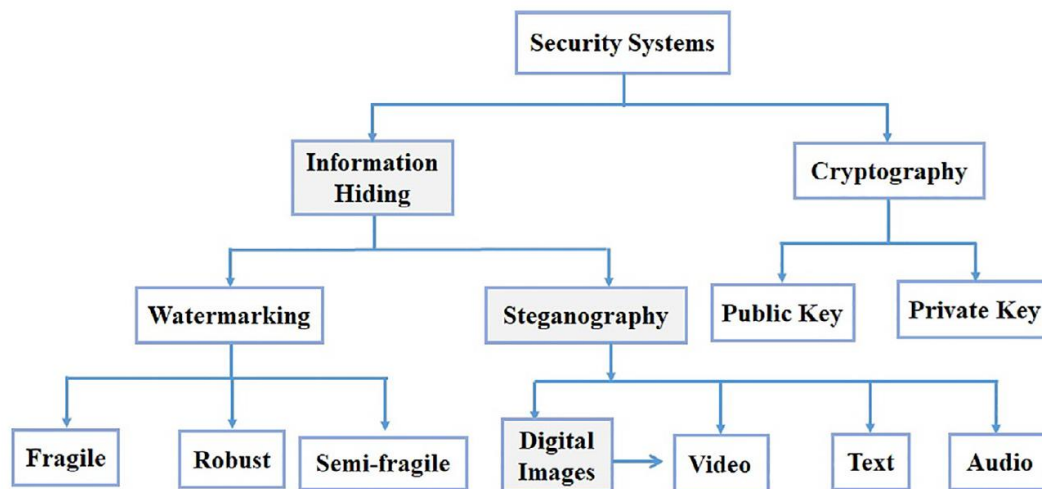


Fig. 1. Basic security system branches [3].

[1]

Steganography finds its application in various fields like- secure communication, copyright protection, covert operations, Military communications, Intelligence operations, Data authentication, Journalism whistleblowing etc.

3. LITERATURE & BACKGROUND

Many revolutionary works have been done in the field of Communication and Information Technology since the Inception of the Internet. But one of the most important factors is Safeguarding of Information. In order to keep messages confidential and Secure many methods were created to Encrypt and Decrypt the data Such as Cryptography. Cryptography was Invented as a technique of securing the secrecy of messages. But sometimes it is not enough to keep only the content of the messages secret, it is also necessary to keep the existence of the messages secret. The technique is Introduced to practice this is called Steganography [2].

3.1 Types of Steganography

1. Text Steganography :- Text steganography consists of hiding text information within text files by embedding the secret data behind specific letters at consistent intervals within the word of the text message. There are several methods for hiding text data :- 1)Format based method; 2)Random and statistical method; 3)Linguistics method [5].

2. Image Steganography:- In Image Steganography, information is exclusively embedded within the visual elements, utilizing images as primary medium for data hiding [2].

The various terms used in image steganography are [7]:-

- **Cover image:-** Cover image is a unique image that can hide data.
- **Information:-** The data that we can conceal within images. The information may be in the form of standard text or an image.
- **Stego image:-** An image containing hidden data is referred to as stego image.
- **Stego key :-** Information can be embedded within cover images and stego images utilizing a key.

3. Audio Steganography:- Audio steganography is practiced by embedding information in Audio files. These methods are used in Audio steganography :- 1) Low bit encoding 2) phase coding; 3) spread spectrum [5].

4. Video Steganography:- Video steganography consists of hiding chunks of secret data in video sequence.

Two types of video steganography are:-

- Hiding information in uncompressed raw video and then compressing it later.
- Hiding data directly into the compressed data stream [5].

5. Network Steganography:- Network steganography consists of hiding information within network control protocols like TCP,UDP,ICMP,IP etc [6].

3.2 Ancient Steganography

Steganography is a technique of concealing information in other information. The word Steganography originated from the Greek words "Stagos" means "Cover" and "Grafia" means "Writing". defining it as "Cover Writing"[1]. In the realm of Image steganography, information is exclusively embedded within the visual elements, utilizing images as a primary medium for data hiding [2].

Steganography has a long history. The term steganography was coined at the end of the 15th century. In ancient times steganography was practiced in many ways like using invisible inks, tattooing on the scalp of slaves, hiding on tablets covered with wax, writing on the stomach of rabbits etc [3]. The earliest evidence of steganography was Herodotus during 484-425 BC. He dictates how his master sent one of his slaves to the Ionian city of Miletus with secret information hidden on his head [1][2]. Vatsyayana, an Indian Philosopher he lived during the second or

third century, wrote in his book how lovers used to communicate secretly at that time. For example, He dictates that lovers send cloves as a message which means he/she wants to meet his/her lover that night.

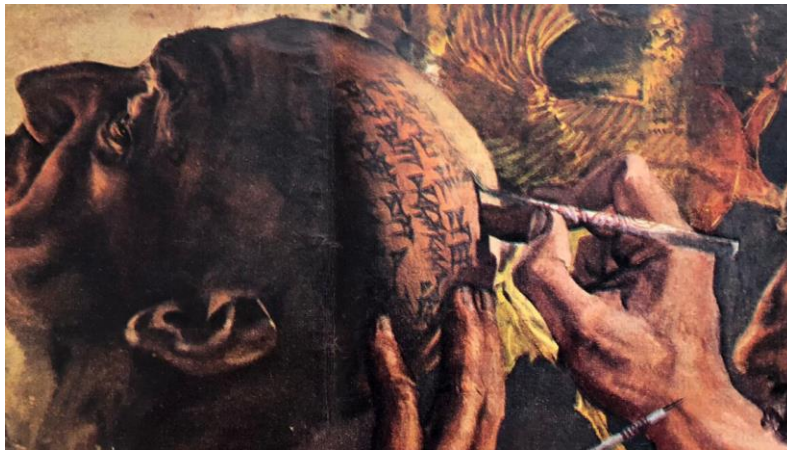


Fig.2 Hiding information on Slaves head [4].

3.3 Modern Era of Steganography

During Earlier days steganography involved concealing secret text into the picture through watermarks. In the Modern era, data is embedded into the file by changing the Bits(Binary Digits) of the file.

In steganography, “Payload”(secret message) is embedded into the “carrier” file. Carrier files are the medium in which the payload is concealed to be transmitted further. Carrier files can be of various forms like image, audio, video or any other digital format capable of holding additional data without visibly altering the content.

Several techniques are employed for steganography to embed the largest payload without altering the original content while maintaining the integrity and functionality of the carrier file. These techniques are further subdivided into Spatial domain and Transform domain.

Steganography Techniques we used in this paper:-

Spatial Domain:- Spatial domain involves hiding confidential data by embedding directly intensity of pixels. In this method pixels values of images changed directly during encrypting data In the spatial domain LSBs(least significant bits) of the cover image are replaced by MSBs(Most significant bits) of the payload file. In the transform domain, the payload and/or the cover image is transformed into the transform domain through DCT(discrete cosine transform), DWT(discrete wavelet transform), DFT(discrete Fourier transform), SSIS(spread Spectrum Image Steganography)etc.

1) Least Significant Bits (LSBs):- Least significant bit is a prevalent technique for embedding data. In this method embedding is practiced by altering the least significant bit of image pixels with corresponding bits from secret data. This LSBs technique involves shuffling the least significant bit (8th bit) of one or more bytes within an image to combine the hidden information. In a particular 24-bit image, which consists of three color bit components—red, green, and blue—each byte stores 3 bits per pixel. For illustration, an 800×600 pixel image can accommodate a total of 1,440,000 bits or 180,000 bytes of hidden data. For instance, using a grid for 3 pixels of a 24-bit image:

(11100101 01011010 10101001)
(00111011 10010110 01011100)

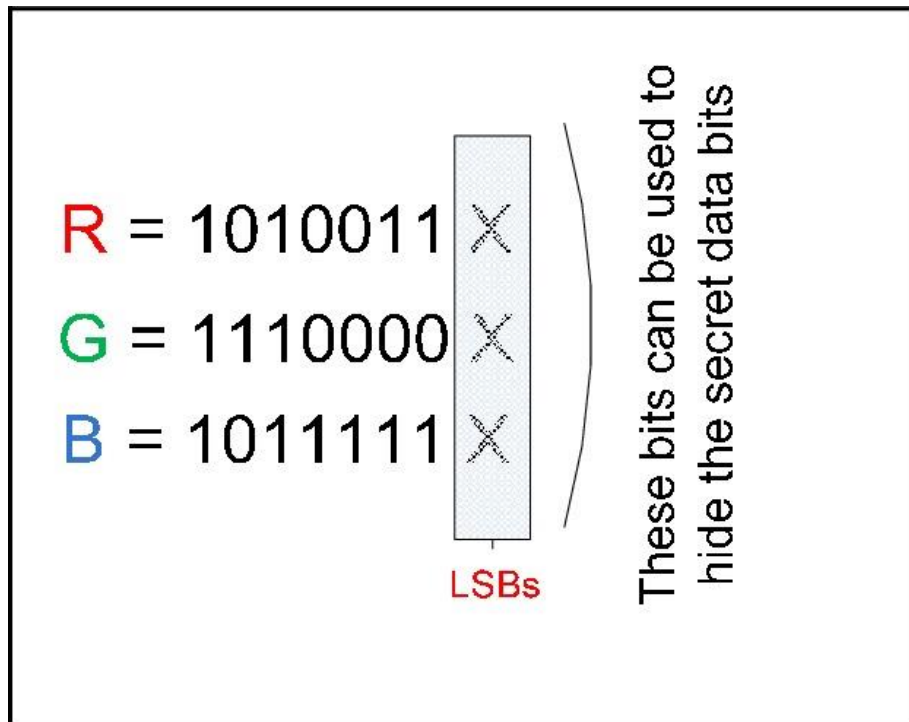
(01101001 11001100 10110110)

Suppose we want to hide the binary representation of the number 123 (01111011) into the least significant bits of this image segment. The modified grid would appear as follows:

(11100100 01011011 10101000)
(00111010 10010111 01011101)

(01101000 11001101 10110111)

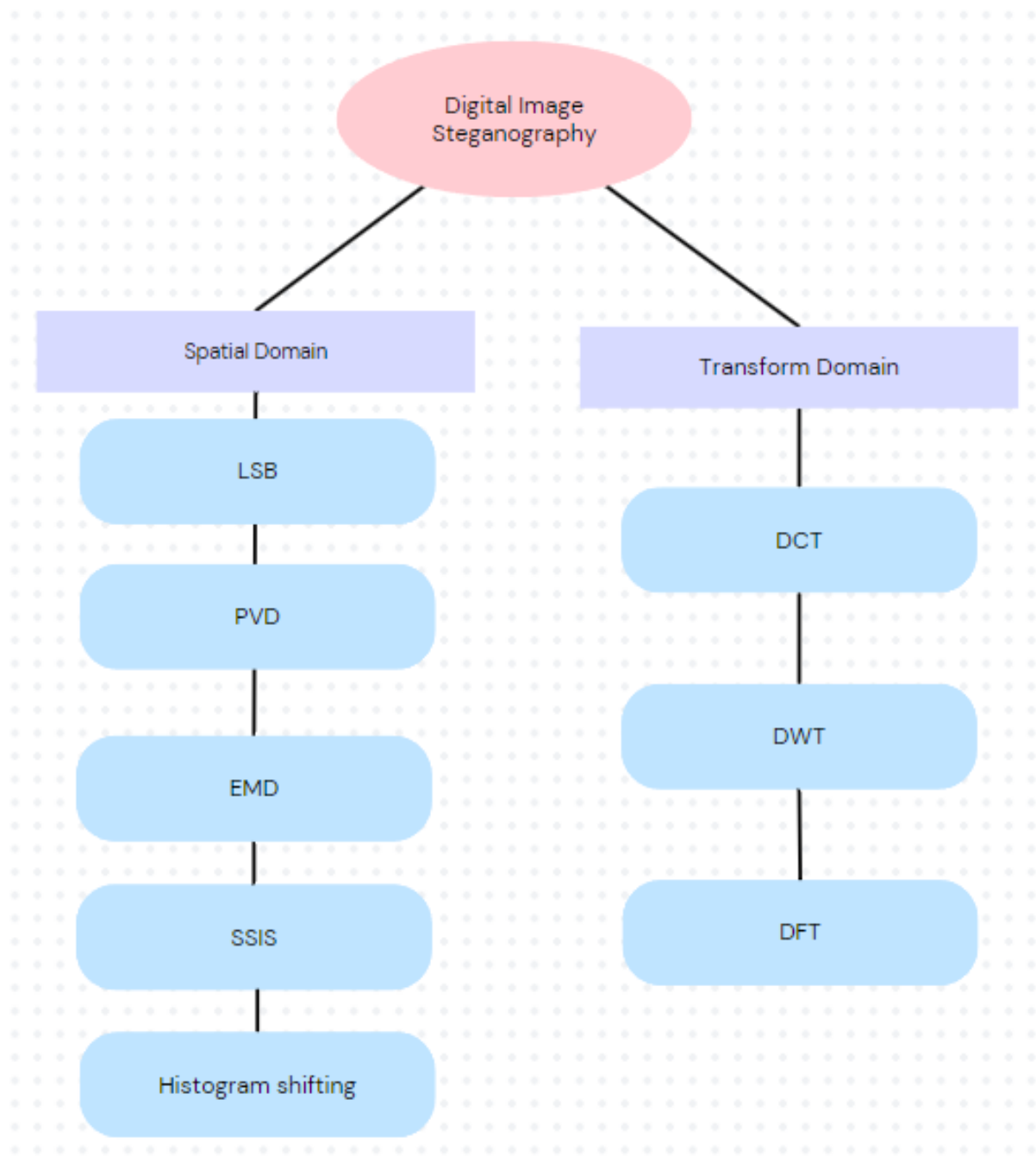
This approach, while apparently straightforward, effectively hides information within the image by altering the least significant bits. It offers a simplistic yet efficient means of embedding information within images without significantly affecting their visual quality [8].



[12]

2) SSIS (Spread Spectrum Image Steganography) :- Spread spectrum image steganography (SSIS) is a technique that hides information within an image without requiring the exact original image for extraction [10]. Instead, it relies on an encryption key for data retrieval. This process involves encoding the data, applying a low-rate error control code, and embedding it within randomly generated noise using a key and modulator. SSIS is commonly represented as additive white Gaussian noise. The encrypted data undergoes quantization to

generate the stego image, which is then transmitted through a secure channel alongside the encryption keys. The decoder reconstructs an estimation of the original cover image using reverse-encoding techniques in image restoration. The effectiveness of the SSIS technique imperceptibility is evaluated using the signal-to-noise ratio (PSNR). An alternative SSIS approach employs chaos-based encryption, utilizing chaotic shift keying for encrypting the secret message signal instead of a pseudorandom noise generator for generating white noise during embedding. Upon reception, chaotic modulation techniques are applied for decoding and data retrieval [9].



4. DATASET

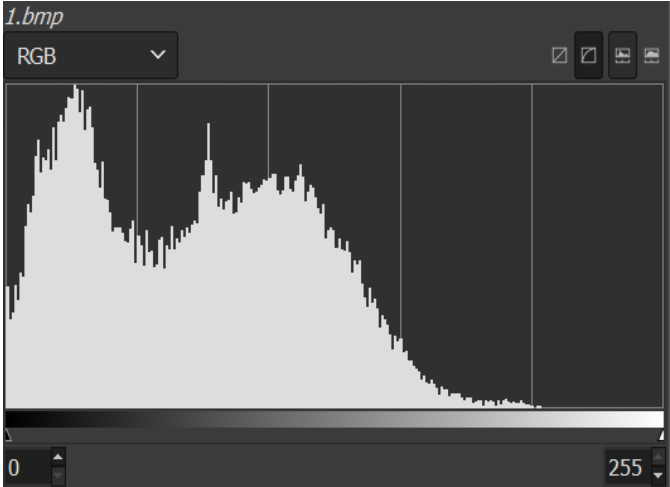
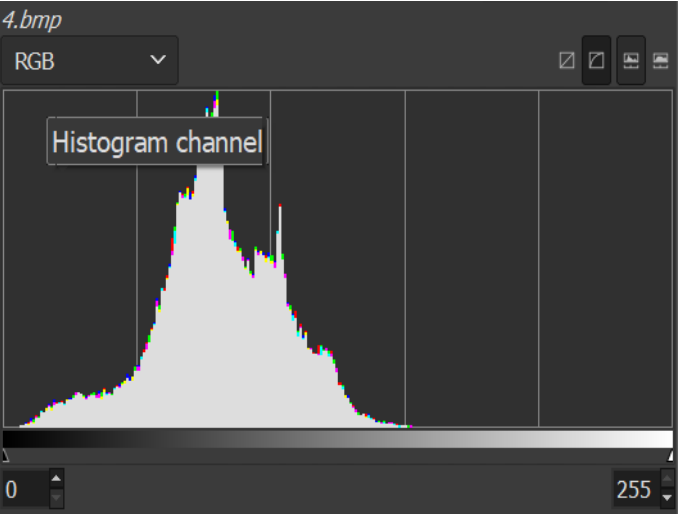
We have taken a dataset of 50 images(25 colored and 25 greyscale). Features of our images are:

- A. 256x256 pixel bitmap images.
- B. Each pixel of our bitmap image comprises only 3 RGB channels.
- C. Bits Per Pixel (BPP) value of our each image is 24.0 [8 pixel to each channel].

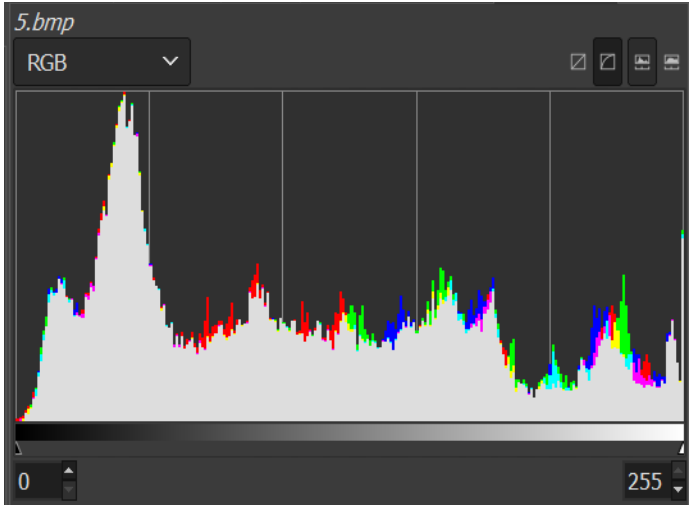
❖ NOTE

BPP or bits per pixel denotes the number of bits per pixel. The number of different colors in an image depends on the depth of color or bits per pixel.

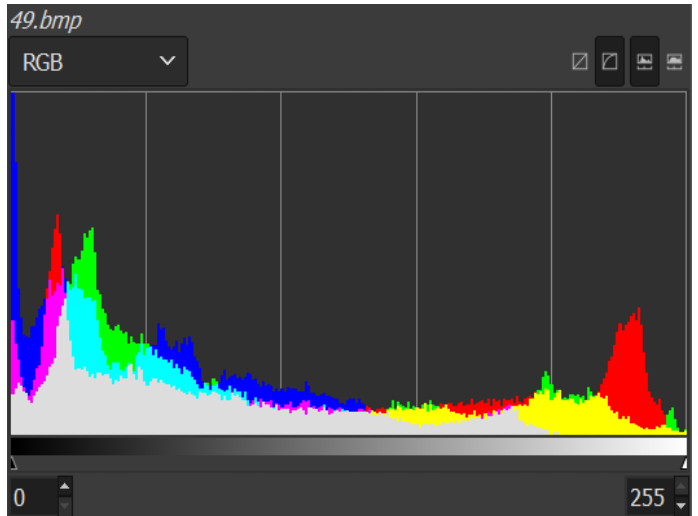
Table-1: Cover images or the original dataset we are taking to hide text messages

S.no.	Cover Images	Histograms
1.		
2.		

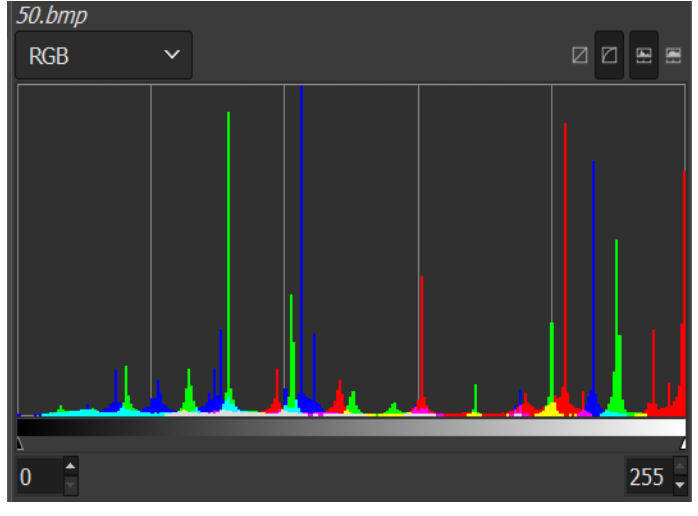
3.



4.



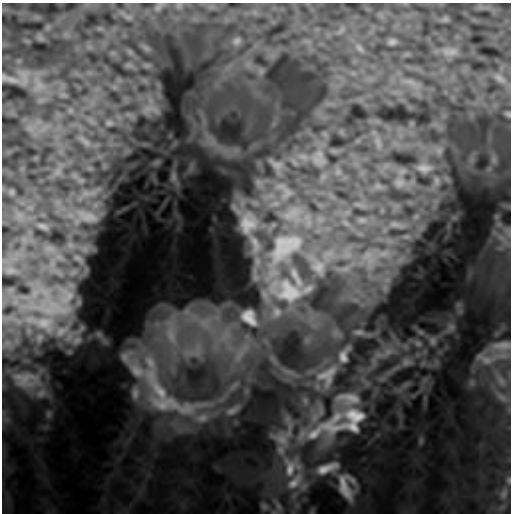
5.



Now, we have divided these images into two sets. In 1st set we have encoded 8KB of data in the images and in 2nd set we have encoded 25KB of that is the images.

[25KB is the highest limit of the payload size of 256x256 pixel bitmap images to be encoded.]

Table-2: 1st set of **8KB** of data encoded in our hybrid method and steganography tool.

S.no.	Stego Tool	Hybrid Method
1.		
2.		

3.



4.



5.



Table-3: 2nd set of **25 KB** of payload encoded in our hybrid method and steganography tool.

S.no.	Stego Tool	Hybrid Method
1.		
2.		
3.		

4.



5.



5. HYBRID METHODOLOGY

Combining the LSB (Least Significant Bit) and SSIS (Spread Spectrum Image Steganography) methods in a hybrid approach involves embedding different parts of the secret message using each method and then combining the results to create a single stego image. Here is a step-by-step guide on how to combine both methods:

Hybrid Image Steganography: Combining LSB and SSIS

1. Secret Message Partitioning:

Divide the secret message into two parts: Part A and Part B.

2. LSB Embedding:

- Implement the LSB method to embed Part A of the secret message into the least significant bits of selected pixel values.
- Convert Part A of the secret message into binary format for embedding.
- Choose the LSB plane for embedding to minimize visual impact.

A. Convert the input text message into a binary format.

B. Append a delimiter ('111111111111110') to indicate the end of the message.

C. Iterate through the image pixels and color channels.

D. Replace the least significant bit of each color channel with the corresponding bit from the binary message.

E. Continue until the entire message is embedded or the delimiter is found.

F. Return the image with the embedded message.

3. SSIS Embedding:

- Implement the SSIS method for spread spectrum embedding.
- Select a suitable carrier signal (e.g., pseudorandom noise sequence) for spreading Part B of the secret message.
- Adjust the spreading parameters for optimal imperceptibility and robustness.

A. Convert the input text message into a binary format.

B. Iterate through the image pixels and color channels.

C. Adjust the pixel values by adding a fraction (alpha) of the corresponding bit from the binary message.

D. Ensure the resulting pixel value is within the valid range [0, 255].

E. Continue until the entire message is embedded.

F. Return the image with the embedded message.

4. Hybridization:

Create a hybrid stego image by combining the LSB-embedded image (with Part A) and the SSIS-embedded image (with Part B).

Use a weighted blending mechanism to combine the two images, ensuring a balance between imperceptibility and robustness.

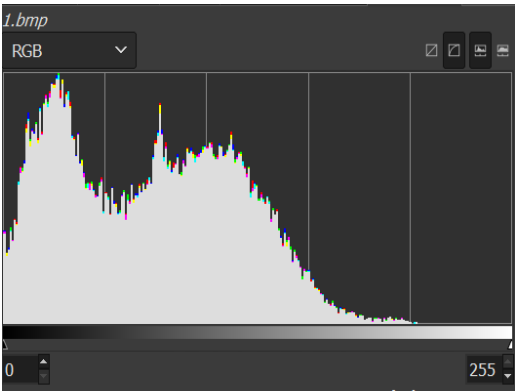
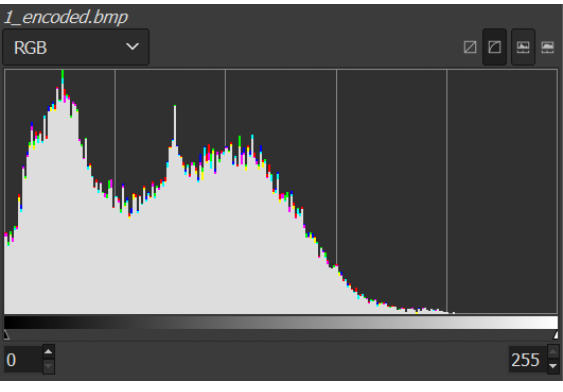
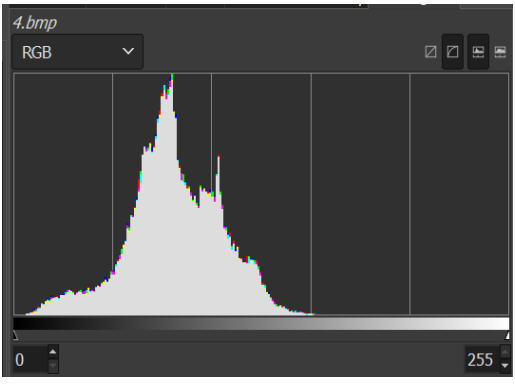
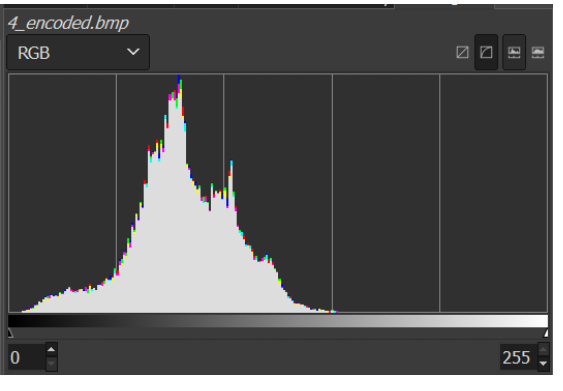
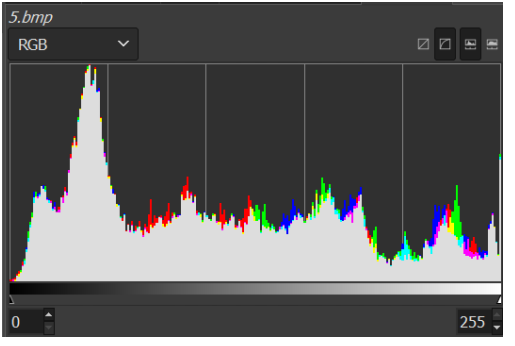
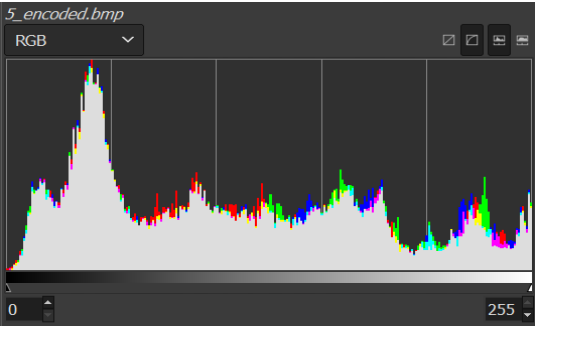
6. EXTRACTION PROCESS

To extract the original message from the stego image, follow the process:

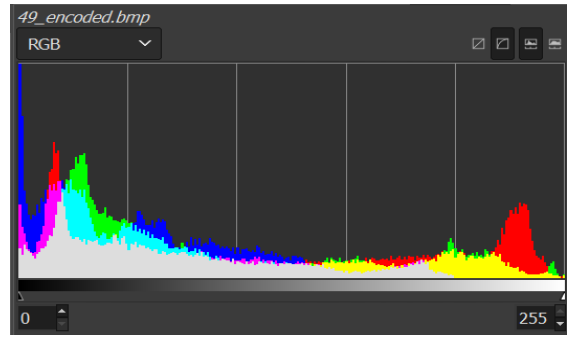
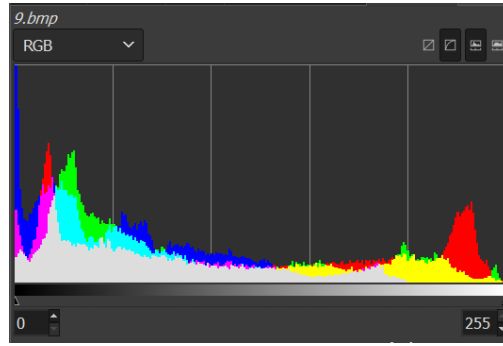
1. Iterates through each character in the text, retrieves the ASCII code for each character, and converts it to an 8-bit binary representation.
2. Iterates through the binary message, extracts 8-bit substrings, converts them to integers, and then converts those integers to ASCII characters. If the ASCII values are within the valid range (0 to 255), they are appended to the decoded text.
3. First, decode the LSB encoded message. Iterates through the pixels of the image, extracts the LSB of each color channel, and appends it to the binary message. The function looks for a delimiter ('111111111111110') in the binary message and removes it to obtain the actual binary message.
4. Then decode the Spread Spectrum part. Iterates through the pixels of the image, extracts the spread spectrum encoded message from each color channel, and appends it to the binary message. The alpha parameter determines the range for extracting the spread spectrum encoded message.
5. Finally, convert the binary message into text and print the decoded message.

7. RESULT

Table-4: Comparing histograms of images of both (our hybrid method and steganography tool) when payload is of **8KB**.

S.no.	Histogram for Stego tool Images	Histogram for our Hybrid Method Images
1.		
2.		
3.		

4.



5.

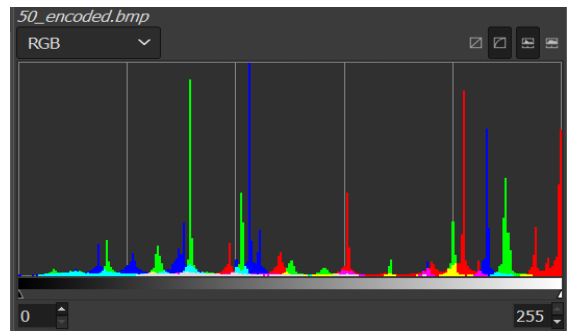
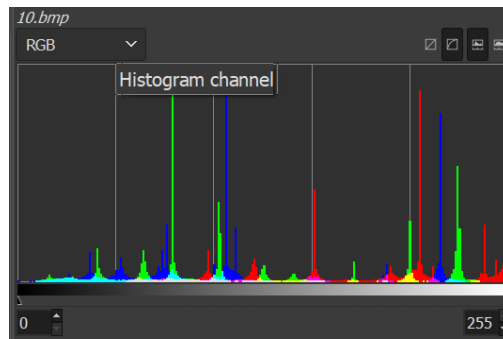
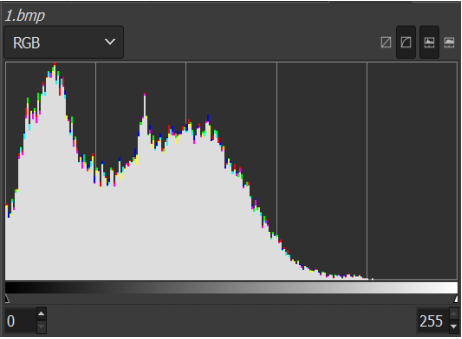
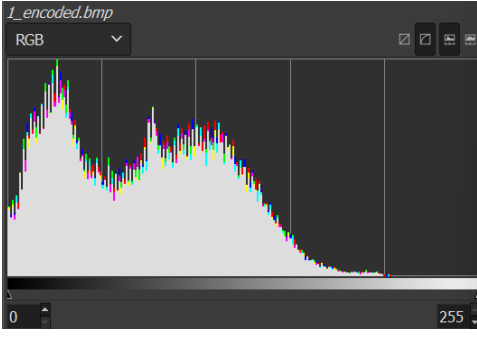
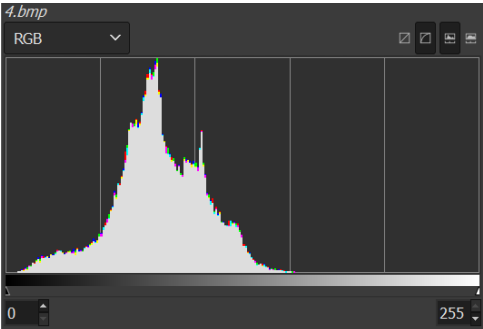
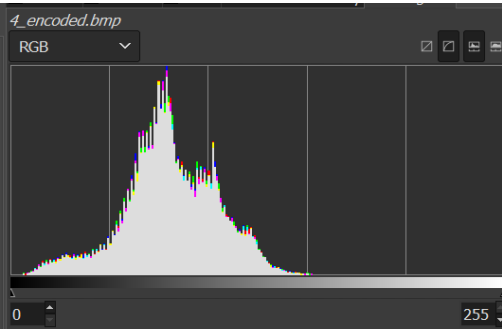
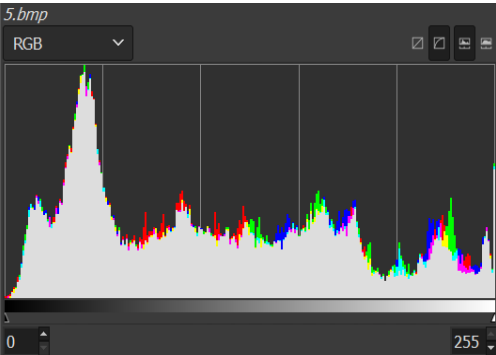
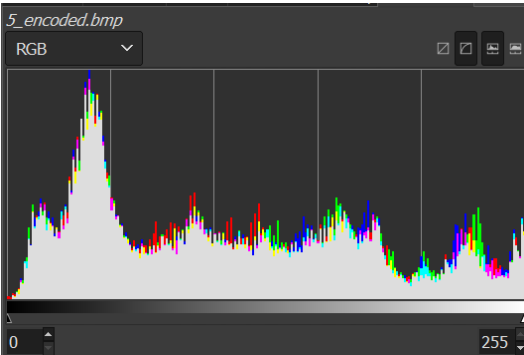
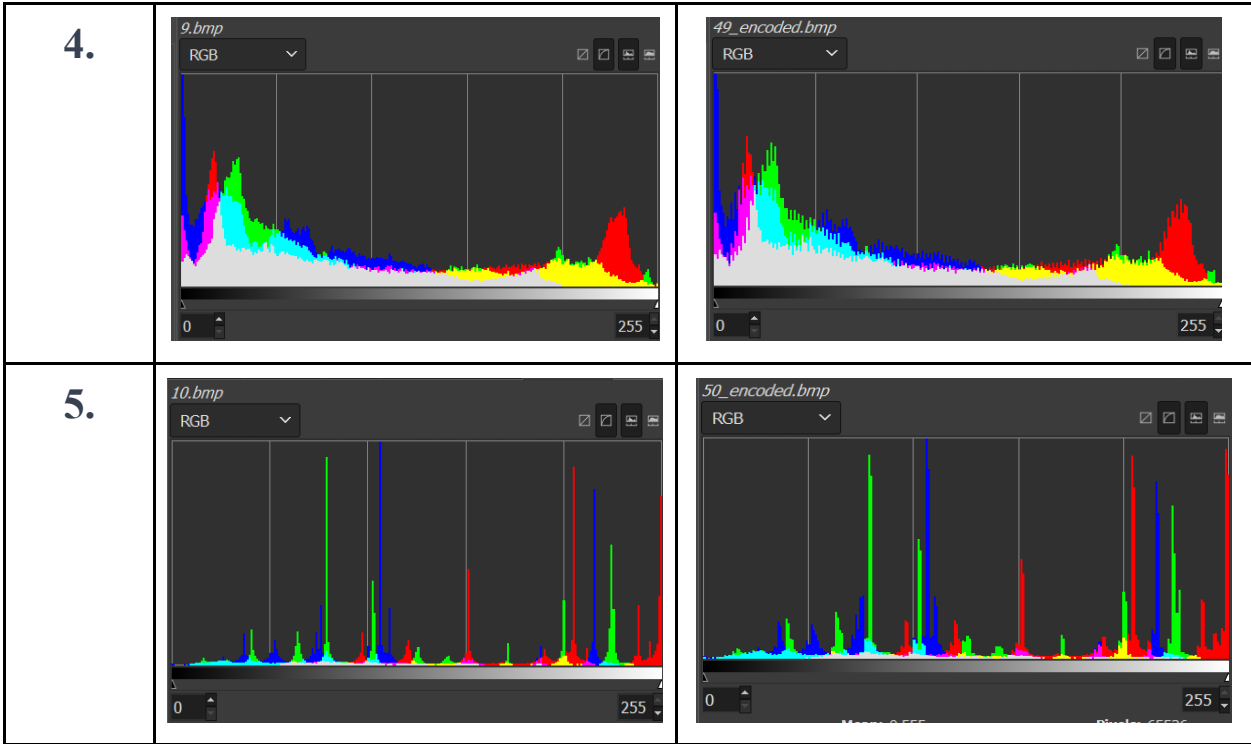


Table-5: Comparing histograms of images of both (our hybrid method and steganography tool) when payload is of **25KB**.

S.no.	Histogram for Stego tool Images	Histogram for our Hybrid Method Images
1.	 The histogram for image 1.bmp, processed by the Stego tool, shows a broad distribution of pixel intensities across the RGB channels. The x-axis represents pixel intensity from 0 to 255, and the y-axis represents frequency. The distribution is relatively smooth with some minor fluctuations.	 The histogram for image 1_encoded.bmp, processed by the Hybrid Method, shows a distribution very similar to the Stego tool result. It features a broad peak across the intensity range, indicating good camouflage of the payload.
2.	 The histogram for image 4.bmp, processed by the Stego tool, shows a more concentrated distribution of pixel intensities compared to image 1. There is a prominent peak in the mid-range intensities.	 The histogram for image 4_encoded.bmp, processed by the Hybrid Method, shows a distribution that closely matches the Stego tool result for image 4, with a clear peak in the mid-range intensities.
3.	 The histogram for image 5.bmp, processed by the Stego tool, shows a highly irregular and noisy distribution of pixel intensities. There are multiple sharp peaks and valleys across the intensity range.	 The histogram for image 5_encoded.bmp, processed by the Hybrid Method, shows a distribution that is significantly smoother and more uniform than the Stego tool result for image 5. It maintains the general shape of the original image's histogram while reducing the noise introduced by the steganography process.



PSNR (Peak Signal to Noise Ratio):

It is a measure used to assess the quality of a denoised or reconstructed image by comparing it to the original, noise-free image. It quantifies the ratio of the maximum possible signal power to the power of unwanted noise, with higher PSNR values indicating better image quality. The metric is often expressed in decibels (dB), providing a straightforward way to evaluate the fidelity of an image after denoising or reconstruction.

$$PSNR = 10 \cdot \log_{10} \left(\frac{L^2}{MSE} \right) = 20 \cdot \log_{10} \left(\frac{L}{\sqrt{MSE}} \right),$$

where L denotes an image's peak signal level which is equal to 255.

MSE (Mean Square Error):

It is a common metric used to quantify the difference between two images or the quality of an image reconstruction. It is a measure of the average squared differences between corresponding pixel values of the original image and the reconstructed or processed image. The formula for Mean Square Error is given by:

$$MSE = \frac{1}{HW} \sum_{i=1}^H \sum_{j=1}^W (P(i, j) - S(i, j))^2,$$

where H and W respectively denote the numbers of rows and columns of the host image, P (i, j) represents the pixel of the host image at the (i, j) position and S(i, j) represents the pixel of the stego image at the (i, j) position.

Table-6: Comparing PSNR values of images from both models and with both payloads i.e. 8 KB and 25 KB

S.no.	Color Channel	8KB Stego Tool	25KB Stego Tool	8KB Hybrid	25KB Hybrid
1.	Red	61.953 dB	57.540 dB	56.045 dB	51.140 dB
	Green	62.009 dB	57.558 dB	55.980 dB	51.139 dB
	Blue	61.848 dB	57,578 dB	55.951 dB	51.183 dB
2.	Red	60.397 dB	56.614 dB	56.042 dB	51.135 dB
	Green	60.467 dB	56.713 dB	56.020 dB	51.184 dB
	Blue	60.419 dB	56.758 dB	56.065 dB	51.141 dB
3.	Red	52.162 dB	48.446 dB	56.017 dB	52.459 dB
	Green	52.501 dB	48.120 dB	56.031 dB	52.445 dB
	Blue	52.337 dB	48.141 dB	55.998 dB	52.333 dB
4.	Red	47.740 dB	45.571 dB	56.059 dB	51.171 dB
	Green	47.724 dB	45.661 dB	56.009 dB	51.143 dB
	Blue	47.411 dB	45.749 dB	55.969 dB	51.142 dB
5.	Red	51.667 dB	48.571 dB	55.984 dB	51.072 dB
	Green	52.124 dB	48.740 dB	55.983 dB	51.113 dB
	Blue	50.975 dB	47,731 dB	56.056 dB	51.023 dB

Table-7: Comparing **MSE** values of images from both models and with both payloads i.e. **8 KB** and **25KB**

S.no.	Color Channel	8KB Stego Tool	25KB Stego Tool	8KB Hybrid	25KB Hybrid
1.	Red	0.0414	0.1145	0.1616	0.5000
	Green	0.0409	0.1140	0.1640	0.5002
	Blue	0.0424	0.1135	0.1651	0.4951
2.	Red	0.0593	0.1417	0.1617	0.5006
	Green	0.0583	0.1385	0.1625	0.4949
	Blue	0.0590	0.1371	0.1608	0.4997
3.	Red	0.3951	0.929	0.1626	0.4992
	Green	0.3655	1.0023	0.1621	0.5028
	Blue	0.3795	0.9366	0.1634	0.5015
4.	Red	1.0939	1.7716	0.1611	0.4966
	Green	1.0980	1.7642	0.1629	0.4997
	Blue	1.1800	1.7303	0.1644	0.4998
5.	Red	0.4429	0.9034	0.1673	0.5079
	Green	0.3987	0.8691	0.1639	0.5032
	Blue	0.5194	1.0963	0.1612	0.4897

8. CONCLUSION

In conclusion, the approach to decode a hidden message from an image encoded using a hybrid combination of the Least Significant Bit (LSB) and Spread Spectrum Image Steganography (SSIS) methods is found out to be effective and comprehensive, successfully revealing the concealed information through the integrated decoding results of both LSB and SSIS techniques. The decoding process successfully extracts the concealed information by combining the results of LSB and SSIS decoding, providing a comprehensive understanding of the embedded message. This hybrid decoding methodology demonstrates an effective fusion of LSB's simplicity and SSIS's resilience, contributing to a more secure and reliable approach to image steganography decoding. The images encoded by this hybrid method is giving consistent and better result for PSNR and SSIS values as compared to the images encoded by the Steganography tool.

9. REFERENCE

1. <https://www.sciencedirect.com/science/article/pii/S002002552200809X>
2. [https://d1wqtxts1xzle7.cloudfront.net/30900669/stegoverview-libre.pdf?1392204634=&response-content-disposition=inline%3B+filename%3DAn overview of image steganography](https://d1wqtxts1xzle7.cloudfront.net/30900669/stegoverview-libre.pdf?1392204634=&response-content-disposition=inline%3B+filename%3DAn+overview+of+image+steganography)
3. <https://www.geeksforgeeks.org/early-evidence-of-steganography>
4. <https://www.tattoolife.com/tattooing-as-a-vehicle-for-secret-messages-in-ancient-greece>
5. [https://d1wqtxts1xzle7.cloudfront.net/61279125/Steganography Techniques A Review Paper.pdf](https://d1wqtxts1xzle7.cloudfront.net/61279125/Steganography+Techniques+A+Review+Paper.pdf)
6. <https://www.kaspersky.com/resource-center/definitions/what-is-steganography>
7. <https://www.simplilearn.com/what-is-steganography-article>
8. [https://www.researchgate.net/profile/Sunny-Behal/publication/275597046 A Survey on various types of Steganography and Analysis of Hiding Techniques/links/565be97408ae1ef92981597f/A-Survey-on-various-types-of-Steganography-and-Analysis-of-Hiding-Techniques.pdf](https://www.researchgate.net/profile/Sunny-Behal/publication/275597046+A+Survey+on+various+types+of+Steganography+and+Analysis+of+Hiding+Techniques/links/565be97408ae1ef92981597f/A-Survey-on-various-types-of-Steganography-and-Analysis-of-Hiding-Techniques.pdf)
9. https://papers.ssrn.com/sol3/papers.cfm?abstract_id=3171494
10. <https://ieeexplore.ieee.org/document/777088>
11. [https://www.researchgate.net/profile/Ashwaq-Alabaichi/publication/337076307 Image steganography using least significant bit and secret map techniques/links/62001cb61674b45977cb3c46/Image-steganography-using-least-significant-bit-and-secret-map-techniques.pdf](https://www.researchgate.net/profile/Ashwaq-Alabaichi/publication/337076307+Image+steganography+using+least+significant+bit+and+secret+map+techniques/links/62001cb61674b45977cb3c46/Image-steganography-using-least-significant-bit-and-secret-map-techniques.pdf)
12. [https://www.researchgate.net/figure/Least-Significant-Bit-Steganography-in-one-Pixel fig4_341879552](https://www.researchgate.net/figure/Least-Significant-Bit-Steganography-in-one-Pixel_fig4_341879552)

10. APPENDIX

1. LSB & SSIS Hybrid method Python Code.

```
from PIL import Image
import os

def text_to_binary(text):
    binary_message = ''.join(format(ord(char), '08b') for char in text)
    return binary_message

def resize_image(image, size=(256, 256)):
    return image.resize(size)

def encode_lsb(image, message):
    binary_message = text_to_binary(message)
    binary_message += '111111111111110'

    data_index = 0
    delimiter_found = False

    for i in range(image.size[0]):
        for j in range(image.size[1]):
            pixel = list(image.getpixel((i, j)))
            for color_channel in range(3):
                if data_index < len(binary_message):
                    pixel[color_channel] = (pixel[color_channel] & ~1) | int(binary_message[data_index])
                    data_index += 1
                else:
                    delimiter_found = True
                    break
            image.putpixel((i, j), tuple(pixel))
            if delimiter_found:
                break
        if delimiter_found:
            break

    return image

def spread_spectrum_encode(image, message, alpha=0.01):
    binary_message = text_to_binary(message)

    data_index = 0
    for i in range(image.size[0]):
```

```

        for j in range(image.size[1]):
            pixel = list(image.getpixel((i, j)))
            for color_channel in range(3):
                if data_index < len(binary_message):
                    # Ensure the resulting pixel value is in the valid range [0, 255]
                    new_value = min(255, max(0, pixel[color_channel] + round(alpha *
int(binary_message[data_index]))))
                    pixel[color_channel] = new_value
                    data_index += 1
                else:
                    break
            image.putpixel((i, j), tuple(pixel))
            if data_index >= len(binary_message):
                break
        return image

def combine_lsb_spread_spectrum(image, message):
    # Resize the image to 256x256 pixels
    resized_image = resize_image(image.copy(), size=(256, 256))

    # Encode the first half of the message using LSB
    lsb_message = message
    lsb_encoded_image = encode_lsb(resized_image.copy(), lsb_message)

    # Encode the second half of the message using Spread Spectrum
    ss_message = message
    ss_encoded_image = spread_spectrum_encode(lsb_encoded_image.copy(), ss_message)

    return ss_encoded_image

def msg(filename):
    with open(filename, 'r') as file:
        msg = file.read()
    return msg

image_path = "cover/image/path"
message = msg("textfile/path")

original_image = Image.open(image_path)

# Combine encoding methods on resized image
encoded_image = combine_lsb_spread_spectrum(original_image, message)

output_folder = "output/folder/path"

```

```

os.makedirs(output_folder, exist_ok=True)

base_name = os.path.splitext(os.path.basename(image_path))[0]
output_path = os.path.join(output_folder, base_name + '_encoded.bmp')
encoded_image.save(output_path)
print(f"Encoded image saved at: {output_path}")

```

2. Decoding Method Code.

```

from PIL import Image

def text_to_binary(text):
    binary_message = ''.join(format(ord(char), '08b') for char in text)
    return binary_message

def binary_to_text(binary_message):
    # Convert binary message to text
    text = ''
    for i in range(0, len(binary_message), 8):
        try:
            # Attempt to convert the binary substring to an integer
            char_code = int(binary_message[i:i+8], 2)
            # Check if the character code is a valid ASCII value
            if 0 <= char_code <= 255:
                text += chr(char_code)
            else:
                # If not a valid ASCII value, stop decoding
                break
        except ValueError:
            # If conversion fails, stop decoding
            break

    return text

def decode_lsb(image):
    binary_message = ''
    delimiter = '111111111111110'

    for i in range(image.size[0]):
        for j in range(image.size[1]):
            pixel = list(image.getpixel((i, j)))
            for color_channel in range(3): # RGB channels
                # Extract the LSB of each color channel and append to the binary message
                binary_message += str(pixel[color_channel] & 1)

    # Find the index of the delimiter in the binary message
    delimiter_index = binary_message.find(delimiter)

```

```

    if delimiter_index != -1:
        # Remove the delimiter and get the actual binary message
        binary_message = binary_message[:delimiter_index]
    else:
        print("Delimiter not found. The hidden message may be incomplete.")

    return binary_message

def spread_spectrum_decode(image, alpha=0.1):
    binary_message = ''
    for i in range(image.size[0]):
        for j in range(image.size[1]):
            pixel = list(image.getpixel((i, j)))
            for color_channel in range(3): # RGB channels
                # Extract the spread spectrum encoded message from each color channel
                binary_message += str(pixel[color_channel] % 2**alpha)

    return binary_message

def decode_combined(image):
    # Decode using LSB
    lsb_binary_message = decode_lsb(image)

    # Decode using Spread Spectrum on the LSB-decoded message
    spread_spectrum_binary_message = spread_spectrum_decode(image)

    return lsb_binary_message, spread_spectrum_binary_message

# Take input from user
encoded_image_path = "img/path"

# Open the encoded image
encoded_image = Image.open(encoded_image_path)

# Decode the hidden message
lsb_message, spread_spectrum_message = decode_combined(encoded_image)

# Convert binary message to text
lsb_decoded_text = binary_to_text(lsb_message)
spread_spectrum_decoded_text = binary_to_text(spread_spectrum_message)

print("Decoded message using the hybrid algorithm:")
print(lsb_decoded_text)

```

3. MSE and PSNR value calculating Python code.

```
import imageio
import numpy as np
from skimage.transform import resize

def mse(image1, image2):
    array1 = np.array(image1)
    array2 = np.array(image2)

    if array1.shape != array2.shape:
        raise ValueError("Images must have the same dimensions")

    mse_values = np.mean((array1 - array2) ** 2, axis=(0, 1))
    return mse_values

def psnr(mse_values, max_pixel_value=255):
    psnr_values = 10 * np.log10((max_pixel_value)**2 / mse_values)
    return psnr_values

# Load images
image_path1 = "original/image/path"
image_path2 = "stegao/image/path"
image1 = imageio.imread(image_path1)
image2 = imageio.imread(image_path2)

# Check and resize images if necessary
if image1.shape != image2.shape:
    # Resize images to the same dimensions
    image1 = resize(image1, (image2.shape[0], image2.shape[1]))

# Calculate MSE and PSNR
mse_values = mse(image1, image2)
psnr_values = psnr(mse_values)

# Print results
for i, psnr_value in enumerate(psnr_values):
    if i == 0:
        print(f"PSNR for RED channel: {psnr_value} dB")
    elif i == 1:
        print(f"PSNR for GREEN channel: {psnr_value} dB")
    elif i == 2:
        print(f"PSNR for BLUE channel: {psnr_value} dB\n")

for i, mse_value in enumerate(mse_values):
    if i == 0:
        print(f"MSE for RED channel: {mse_value}")
    elif i == 1:
        print(f"MSE for GREEN channel: {mse_value}")
    elif i == 2:
        print(f"MSE for BLUE channel: {mse_value}\n")
```